Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática
AEDS III - Trabalho Prático
Tema: Sistema de Gerenciamento de Biblioteca

–

Componentes do Grupo: Caroline Freitas Alvernaz, João de Jesus e Avila, Júlia Rodrigues Vasconcellos Melo, Priscila Andrade de Moraes e Sirius Victor Zevallos.
Repositório (GitHub): https://github.com/prizinh4/biblioteca/tree/main
Vídeo Explicativo (Youtube): https://www.youtube.com/watch?v=I3weX7z7Mcs

Documentação do Projeto

Nota: Alterações para Fase 2
A Fase 2 introduz modificações relevantes em relação à modelagem e implementação originalmente propostas, incluindo:

- Alteração no RF01, CRUD de Livros, que agora considera que um livro pode possuir múltiplos exemplares físicos associados; no RF04, que passa a visar registro e gerenciamento de empréstimos dos Exemplares, e não dos Livros; e no modelo de Dados, introduzindo a entidade Exemplar, e removendo a entidade associativa Item_Emprestimo.
- Alteração no RF03, que continua integrado ao CRUD de Livros, porém com tratamento de atributos multivalorados serializados.
- Adição do RF07, que permite a listagem de todos os exemplares, dado o título do Livro.
- Adição do RNF04, que busca garantir que o sistema evite varreduras sequenciais completas; e do RNF05, Integridade Referencial, que busca garantir consistência na relação entre Exemplares e Livros.

1. Descrição do Problema

O sistema visa gerenciar as operações de uma biblioteca, permitindo o controle de suas operações através do CRUD de Livros, Autores, Leitores e Empréstimos. Para garantir a persistência, os dados serão armazenados diretamente em arquivos binários no HD, sem o uso de SGBDs. Os arquivos contarão com cabeçalho contendo informações de controle, e utilizarão a técnica de lápide para controle de exclusão lógica.

2. Objetivos do Trabalho

- Desenvolver um sistema que permita o CRUD de Livros, Leitores, Autores e Empréstimos.

- Garantir persistência direta em arquivos binários no HD com controle de exclusão lógica (lápide).
- Implementar relacionamentos 1:N e N:N e utilizar diversos tipos de dados (Data, Real, String e String Multivalorada).

3. Requisitos Funcionais

- RF01: Cadastrar, consultar, atualizar e excluir (CRUD) Livros, considerando associação entre Livros e Exemplares.
- RF02: Cadastrar, consultar, atualizar e excluir (CRUD) Leitores.
- RF03: Gerenciar informações de autores e categorias integradas ao cadastro de livros.
- RF04: Registrar e gerenciar Empréstimos de exemplares de livros para os leitores.
- RF05: Listar registros ativos de qualquer entidade.
- RF06: Cadastrar, consultar, atualizar e excluir (CRUD) de Exemplares de Livros, onde cada exemplar representa uma instância física de um determinado livro.
- RF07: Consulta de Exemplares por Livros, permitindo listagem de todos os exemplares associados a um determinado livro.

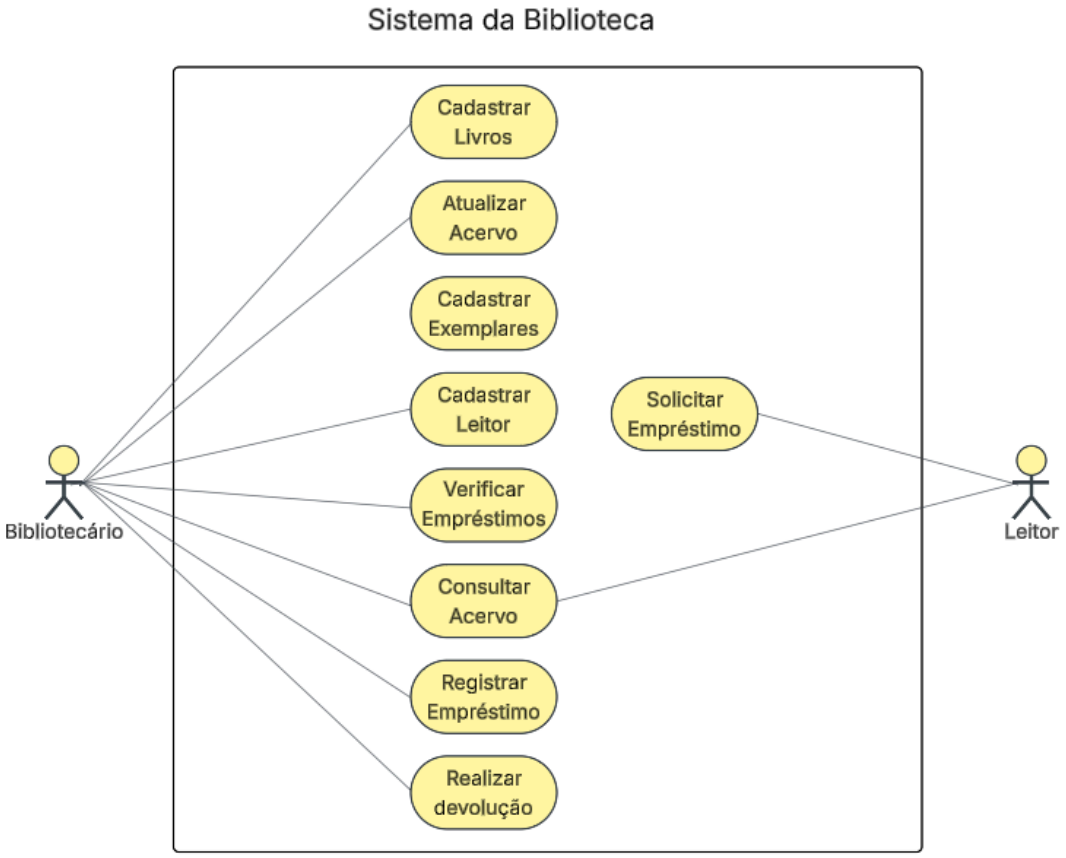
4. Requisitos Não Funcionais

- RNF01: O backend deve ser implementado persistindo dados em arquivos binários.
- RNF02: A interface final (após a Fase 1) não poderá utilizar console, devendo possuir interface mínima em HTML/CSS (Front-end web ou interface gráfica).
- RNF03: O sistema deve seguir o padrão de arquitetura MVC + DAO.
- RNF04: O sistema deve garantir acesso eficiente aos dados por meio de estruturas de indexação (índices diretos e hash extensível).
- RNF05: O sistema deve garantir validação da FK Livro na inserção de Exemplares, exclusão lógica em cascata e consistência em atualizações do relacionamento.

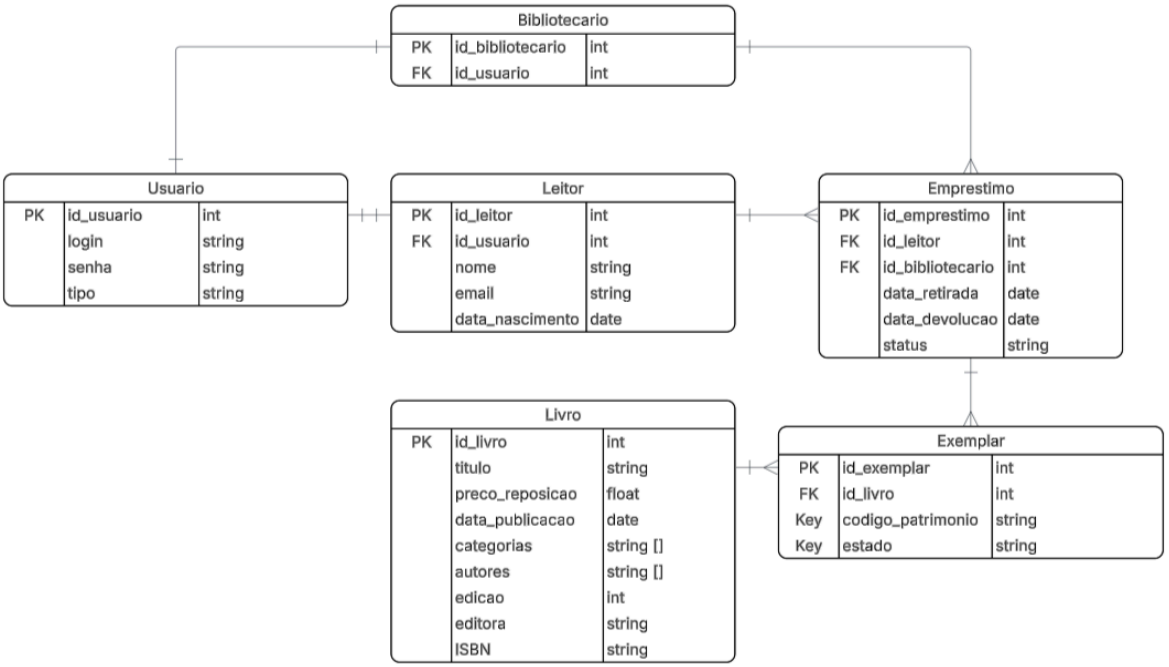
5. Atores

- Bibliotecário (Administrador): Realiza a gestão do acervo (Livros e Autores) e gerencia os cadastros de Leitores.
- Leitor: Usuário final que realiza o empréstimo de livros.

6. Diagrama de Caso de Uso



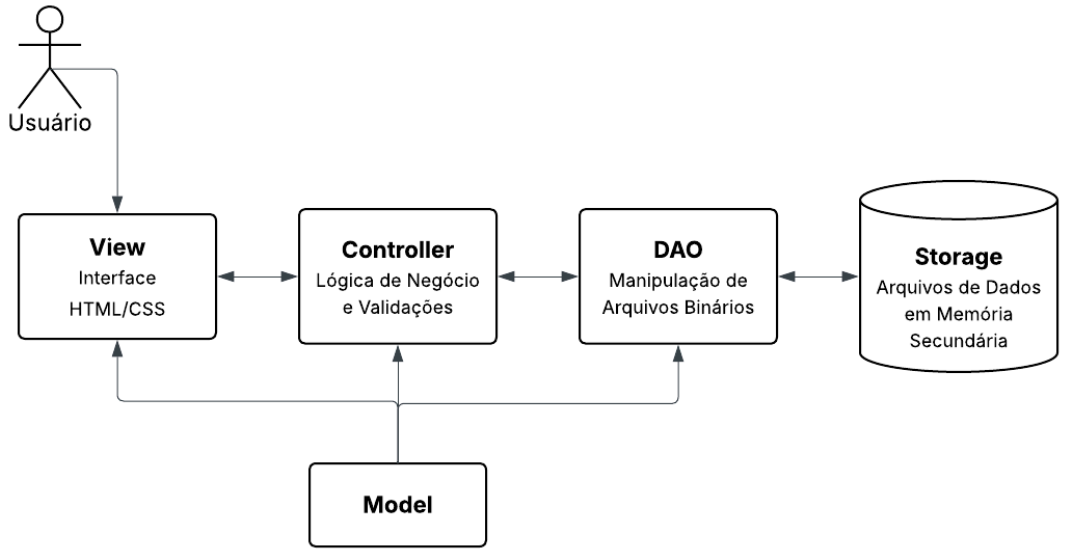
7. Diagrama Entidade-Relacionamento



8. Arquitetura Proposta

O sistema seguirá o padrão MVC + DAO, onde:

- Model: Classes de domínio do sistema (Livro, Leitor, Autor, Emprestimo).
- DAO: Classes responsáveis por ler e escrever diretamente nos arquivos binários (.db), manipulando o cabeçalho e a exclusão lógica (lápide).
- Controller: Classes que gerenciam as regras de negócio (ex: não permitir empréstimo se o leitor tiver multas).
- View: Interface gráfica ou web (HTML/CSS) para interação com o usuário.



O sistema utiliza uma arquitetura em camadas para garantir a separação de responsabilidades e facilitar a manutenção:

- **View:** Camada de interação com o usuário. Inicialmente implementada via Interface Web (HTML/CSS), captura as entradas do usuário e exibe os resultados.
- **Controller:** Atua como intermediária. Ela recebe as solicitações da View, aplica as regras de negócio (como verificar se um livro está disponível antes de realizar o empréstimo) e coordena as chamadas para a camada de persistência.
- **DAO (Data Access Object):** Camada responsável exclusivamente pelo acesso aos dados. É nela que será implementada a lógica de RandomAccessFile para abrir os arquivos .db, gerenciar o ponteiro do arquivo (seek), ler/escrever o cabeçalho e realizar a exclusão lógica via lápide.
- **Model:** Contém as classes de domínio (Livro, Leitor, Exemplar, Empréstimo, Usuário). Essas classes representam a estrutura dos dados que são transferidos entre as camadas e espelham os registros armazenados nos arquivos binários.
- **Persistência:** Os dados são armazenados diretamente no sistema de arquivos do computador em formato binário, garantindo a permanência das informações sem o uso de bancos de dados externos.

Respostas Formulário do Projeto

a) Qual a estrutura usada para representar os registros?

- Registros variáveis em arquivo binário com lápide, tamanho e campos serializados.

b) Como atributos multivalorados do tipo string foram tratados?

- Em Livro, atributos como autores/categorias ficam em string única delimitada por usuário; serialização UTF-8 com tamanho.

c) Como foi implementada a exclusão lógica?

- Campo lápide (boolean) no início do registro. true indica removido.

d) Além das PKs, quais outras chaves foram utilizadas nesta etapa?

- livroId (FK em Exemplar) para relacionamento 1:N.
- ISBN e código de patrimônio são tratados como únicos por regra de validação.

e) Como a estrutura (hash) foi implementada para cada chave de pesquisa?

- PKs: índice direto (id -> posição).
- Relacionamento 1:N: hash extensível (livroId -> ponteiro cabeca) + lista encadeada de filhos.

f) Como foi implementado o relacionamento 1:N? Explique a lógica da navegação entre registros e integridade referencial.

- Ao inserir exemplar, grava-se um novo nó em livro_exemplar.rel apontando para a cabeça antiga da lista do livro.
- O hash atualiza a cabeça para o novo no.
- Para navegar, busca-se cabeça via hash e percorre-se os nos lendo cada exemplar_Id.
- Integridade: só insere um exemplar se o livro existir; exclusão de livro faz cascata lógica nos filhos.

g) Como os índices são persistidos em disco (formato, atualização, sincronização com os dados)?

- IndiceDireto grava pares (int id, long pos) em arquivo binario.
- A cada create/update/delete no DAO, o índice é atualizado imediatamente.
- No hash extensível, diretório e buckets são regravados após split, inserções e remoções.

h) Como está estruturado o projeto no GitHub (pastas, módulos, arquitetura)?

- Raiz com App.java, pasta src/ separada em Model, DAO, controller, e diretório “data” para persistência.
- Arquitetura em camadas com responsabilidade separada.